

«Talento Tech»

# React JS

Clase 03



## Clase N° 3 | Layout en React

### Índice

#### Clase N° 3 | Layout en React

##### Estructura de nuestro proyecto

¿Cómo Organizo mis Carpetas?

##### Estructura y composición de componentes

Repaso y preparación del proyecto

##### Creación del Layout

Maquetando la app: componentes de layout

header y footer.

Layout

##### Aplicando estilo: CSS Global vs. CSS Modules

CSS Modules

CSS Global

##### Segunda fase del proceso para sumarte a Talento Lab

##### Ejercicio práctico:

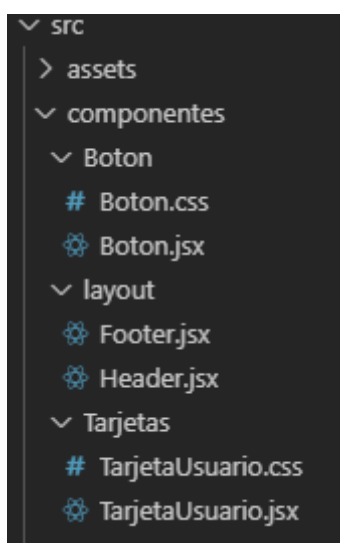
### Objetivos de la clase:

- Aprender a estructurar un proyecto React desde cero.
- Crear componentes reutilizables que representen partes fundamentales de una interfaz.
- Aplicar estilos a nuestros componentes
- Comprender cómo ensamblar y renderizar estos componentes en el navegador.

## Estructura de nuestro proyecto

### ¿Cómo Organizo mis Carpetas?

Una vez que tenemos nuestro proyecto hecho, y con varios componentes creados, nos damos una idea de que a medida que crece nuestra página, crece la cantidad de componentes. Por eso a medida que un proyecto empieza a tener muchos componentes, el orden es fundamental. Una estructura muy común y recomendada es la siguiente:



Dentro de nuestra carpeta src creamos una carpeta llamada componentes, y dentro de ella, creamos una carpeta por cada componente, donde contendrá su componente jsx y su respectivo css.

#### Buenas prácticas:

- **Una carpeta por componente:** Si un componente tiene su propio CSS o archivos relacionados, es una buena idea meter todo en su propia carpeta.
- **Nombres en PascalCase:** Los archivos de componentes siempre con la primera letra en mayúscula (Boton.jsx, TarjetaUsuario.jsx).



**Vite + React**



## Estructura y composición de componentes

### Repaso y preparación del proyecto

Ya sabemos que son los componentes, como se estructuran y cómo podemos organizar nuestro directorio para no marearnos cuando tengamos nuestra pagina mas avanzada.. Ahora vamos a dar un paso más allá y aprenderemos a **estructurar una aplicación completa**, componiendo esos pequeños componentes para formar el layout de una página web.

¡No crearemos un proyecto nuevo! Vamos a seguir trabajando sobre el mismo proyecto de Vite que usamos en las clases anteriores.

1. Abrió tu proyecto en Visual Studio Code.
2. Iniciá el servidor de desarrollo: Abrió una terminal integrada y ejecutá `npm run dev` para ver tu aplicación en el navegador.
3. Limpiá tu `App.jsx`: Vamos a borrar los ejemplos de la clase pasada dentro de la función `App` para tener un lienzo limpio donde empezar a construir nuestro layout.

Tu archivo `/src/App.jsx` debería verse así para empezar:

```
import './App.css';

function App() {
  return (
    <div>
      <h1>Mi Aplicación</h1>
    </div>
  );
}

export default App;
```

### Estilos básicos:

Si queremos darle estilo a alguna etiqueta de nuestro componente, podemos usar la palabra `style` como propiedad del tag HTML para aplicarle css en línea, o aplicarlo con variables como usamos en clases anteriores. La sintaxis sería la siguiente:

```
<H2 style={{ backgroundColor: "#8DE2D6", color: "white" }}>
```

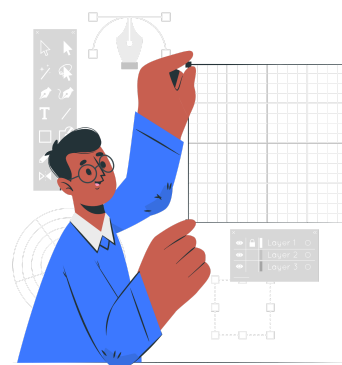
con doble llaves para tomar el par propiedad:valor.

## Creación del Layout

### Maquetando la app: componentes de layout

Como se vió en el curso anterior, toda página web comparte una estructura básica: un encabezado, un contenido principal y un pie de página. En React, en lugar de tener todo en un index.html, creamos componentes reutilizables para cada una de estas partes.

- **Layout.jsx:** Envuelve todo el contenido que queremos mostrar.
- **Header.jsx:** Contendrá el logo, el título principal, etc.
- **Navbar.jsx:** La barra de navegación, que a menudo va dentro del Header.
- **Main.jsx:** Un contenedor para el contenido principal y cambiante de cada página.
- **Footer.jsx:** El pie de página con links, copyright, etc.



Esto no solo es más ordenado, sino que nos permite, por ejemplo, usar el mismo Header y Footer en todas las páginas de nuestra aplicación sin repetir código.

### header y footer.

Vamos a crear dos componentes nuevos para darle forma a nuestro sitio. Crearemos estos dos primeros componentes en su respectiva carpeta con exportación por default.

#### Header.jsx

Este componente representa la cabecera de nuestro sitio web. En el vamos a comenzar mostrando el título de nuestro sitio web.

```
function Header() {
  return (
    <header style={{ backgroundColor: "#8DE2D6", padding: "10px",
    textAlign: "center", color: "white" }}>
      <h1>Bienvenidos a mi App React</h1>
    </header>
  );
}
export default Header;
```

Estamos aplicando CSS en línea directamente al componente (después lo cambiaremos) lo que nos permite agregar un poco de estilo a nuestro página.

## Footer.jsx

Este componente actúa como el pie de página del sitio. Al igual que el header, vamos a agregarle estilo en línea y vamos a exportarlo por default.

¡Completá el footer como a vos mas te guste!

```
function Footer() {
  return (
    <footer style={{ backgroundColor: "#8DE2D6", padding: "10px",
textAlign: "center", marginTop: "20px", color:"black" }}>
      <p>&copy; 2025 - Mi Aplicación React</p>
    </footer>
  );
}
export default Footer;
```

## Layout

Volvamos a la prop children, recordemos que es una prop automática que contiene **todo lo que esté entre la etiqueta de apertura y cierre** de un componente. En este caso nos es perfecta para el componente de maquetado. Podemos crear un componente Layout que envuelva todo el contenido que queramos mostrar.

Para eso vamos a importar e implementar el Header.jsx y Footer.jsx que hicimos anteriormente. Recordá que lo importamos de esta manera porque hicimos un export default. Si hiciste una exportación nombrada, vas a tener que modificar la forma de importar el componente.

```
// En /components/Layout/Layout.jsx
import Header from './Header';
import Footer from './Footer';

// Todo lo que pongamos dentro de <Layout> en App.jsx será el "children".
export function Layout({ children }) {
  return (
    <div>
      <Header />
      <main>
        {children}
      </main>
      <Footer /> // Continúa abajo!
    </div>);}
```

En este componente Layout, agregamos el Header y el Footer lo que nos permite modificarlos sin necesidad de tocar el Layout u otro contenido de nuestra página.

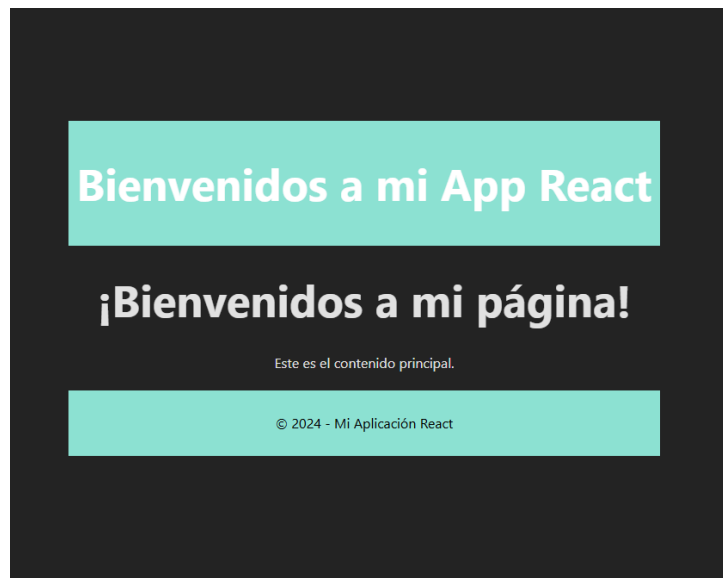
Luego, en App.jsx, lo usamos de esta manera:

1. Lo importamos según corresponda nuestra exportación
2. Dentro de las llaves de apertura y cierre de <Layout>, incorporamos la prop children.

```
// /src/App.jsx
import './App.css';
import { Layout } from './componentes/layout/Layout';
function App() {
  return (
    <Layout>
      {/* Todo lo que pongamos acá adentro irá donde estaba {children} */}
      <h1>¡Bienvenidos a mi página!</h1>
      <p>Este es el contenido principal.</p>
    </Layout>
  );
}
export default App;
```

## Vista previa

Por el momento, nuestra página se verá así:



## Aplicando estilo: CSS Global vs. CSS Modules

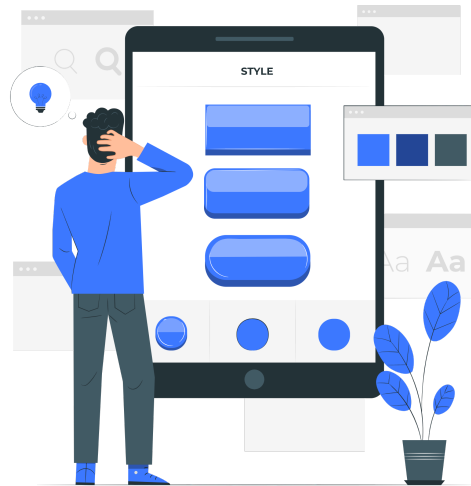
Ahora que tenemos la estructura, vamos a profundizar en los estilos.

Para aplicarle estilos a nuestra página, vamos a ver dos formas distintas:

1. CSS Modules
2. CSS Global

### CSS Modules

La forma recomendada para darle estilo a nuestros componentes y además evitar que los estilos de un componente se pisen con los de otro, usamos **CSS Modules**. Es la mejor práctica y nos salva de muchos dolores de cabeza.



#### ¿Cómo funciona?

1. Elijamos un componente que tengamos creado, por ejemplo Header.jsx y si tiene aplicado css en línea, lo copiamos y lo borramos de este archivo. (con etiqueta style y todo)
2. Creá un archivo de estilos con la extensión **.module.css** en la misma carpeta del componente, layout en nuestro caso, y pegá el css copiado anteriormente.

```
// /componentes/layout/Header.module.css
.header{
  background-color:#8DE2D6;
  padding:10px;
  text-align:center;
  color:"white";
}
```

Prestá atención a la sintaxis de CSS. Es distinta a la sintaxis del estilo inline que podemos aplicar en cada componente.

Ahora vemos que si vamos a nuestra página, no vamos a tener estilo para este componente. Lo que nos falta es importarlo.



En tu componente elegido, importá los estilos como un objeto y usalo de la siguiente manera:

Si en tu archivo `.module.css` creaste una clase llamada `.header`, entonces la forma de usarlo es llamando a esa clase desde el archivo styles **`className={styles.header}`**.

Creá y aplicá otros estilos para el `h1` de esa página.

```
// /componentes/layout/Header.jsx
import styles from './Header.module.css'
function Header() {
  return (
    <header className={styles.header}>
      <h1>Bienvenidos a mi App React</h1>
    </header>
  );
}
export default Header;
```

La gran ventaja: La clase `.header` de este componente nunca chocará con otra clase `.header` que puedas tener en otro archivo. Cada estilo es local a su componente y los CSS Modules generan nombres de clase únicos para evitar colisiones

## CSS Global

Cuando creamos un proyecto con Vite, nos encontramos con dos archivos CSS principales: **`index.css` y `App.css`**. Aunque ambos aplican estilos de forma global, tienen roles distintos.

**`index.css`:** Este archivo es importado en `main.jsx`, que es el punto de entrada de toda tu aplicación. Por lo tanto, los estilos que ponés acá son los de **más alto nivel**, los que afectan al documento entero.

### ¿Para qué se usa?

- Lo usamos para aplicar estilos en `<html>` y `<body>`, como el color de fondo general, la tipografía por defecto para toda la página, `margin: 0`, `box-sizing`, etc.
- **Reseteos de CSS (Resets):** Para que tu página se vea igual en todos los navegadores, eliminando los estilos que traen por defecto.
- **Variables CSS Globales:** Es el lugar ideal para definir variables de color, espaciado, etc., que vas a usar en todo el proyecto.

**App.css:** Este archivo se importa directamente en App.jsx. Su propósito es darle estilo al componente App y a los elementos principales que contiene.

### ¿Para qué se usa?

- Estilos para la maquetación principal de la app (el div con className="App", por ejemplo).
- Clases de utilidad generales que quieras tener a mano.
- Estilos para elementos que solo existen dentro de App.jsx, como el header o footer si los tenés ahí.

Estos estilos se aplican a todo el proyecto. Son útiles, pero hay que tener cuidado de no definir clases muy genéricas que puedan chocar entre sí.

### Aplicando estilos globales

Vamos a modificar nuestro archivo **index.css** para controlar todos los aspectos de nuestro sitio web. Para eso vamos a eliminar su contenido y vamos a aplicarle las normas generales de nuestra aplicación:

```
*{
  margin:0;
  padding: 0;
  box-sizing: content-box;
}
body {
  font-family: 'Helvetica', sans-serif;
  margin: 0;
  background-color: #f4f4f4;
}
h1, h2, h3, p {
  color:black;
  font-size: 16px;
}
```

### ¿Por qué al cambiar index.css no cambia todo?

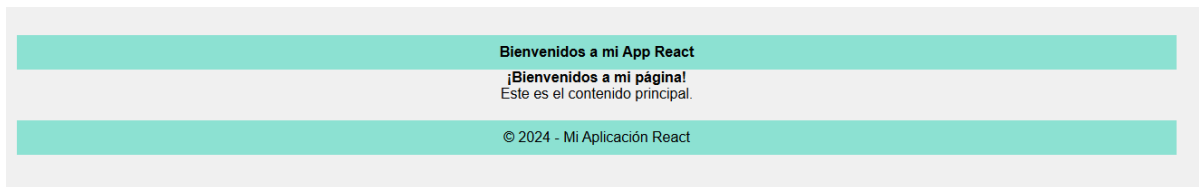
Acá entra en juego un concepto fundamental que tenemos que recordar de CSS: **la especificidad y la cascada**.

1. Primero, el navegador aplica sus estilos por defecto.
2. Luego, vos aplicás los tuyos con index.css, por ejemplo background-color: #f4f4f4.
3. Después, viene una capa más específica con App.css. Por ejemplo, si en App.css tenés una clase .contenedor-especial con background-color: white, cualquier etiqueta que tenga esa clase será blanco, tapando el gris que definiste en index.css.

Con estos cambios puedo ver que cambió mucho el diseño de la aplicación. Lo último que te propongo, es desde el inspector del sitio web, encontrar quien tiene padding y sacárselo.

### Vista previa

Por el momento nuestra página, sin el padding se verá de esta manera :



Continuaremos la siguiente clase trabajando sobre este archivo.

---

## Reflexión final

Aprendimos a estructurar un proyecto React desde cero, enfocándonos en la creación de componentes reutilizables que representan las partes fundamentales de una interfaz de usuario, como el Header, Navbar y Footer. Estos componentes nos permiten modularizar el código y mejorar su mantenibilidad, además de fomentar la reutilización en diferentes contextos.

La práctica de ensamblar estos componentes en App.jsx no sólo consolidó conceptos básicos de React, sino que también nos mostró cómo trabajar de manera colaborativa en proyectos más grandes. Este enfoque facilita la organización del proyecto y establece una base sólida para desarrollar aplicaciones más complejas en el futuro.

## Segunda fase del proceso para sumarte a Talento Lab



En este segundo ejercicio práctico te desafiamos a demostrar tu creatividad y habilidades técnicas para estructurar y diseñar una interfaz dinámica y funcional utilizando React. Prepárate para este reto, que evaluará tu capacidad para trabajar con componentes, props y estilos.



### Ejercicio práctico:

#### Maquetando la home de tu E-commerce

**Objetivo:** Crear la estructura visual de la página principal para una tienda online, aplicando los conceptos de componentización y estilos vistos en clase.

#### Consignas:

- 1. Definir la identidad visual:**
  - Elegí una paleta de colores que te guste y que creas que se relaciona con la marca o el tipo de productos de tu e-commerce.
- 2. Construir el Layout principal:**
  - Dale estilo al Header y al Footer para que sean visualmente atractivos y coherentes con tu paleta de colores.
  - Agregá una barra de navegación (<nav>) simple dentro del Header con enlaces de ejemplo (Ej: "Inicio", "Productos", "Contacto", "Carrito").
- 3. Crear el componente de producto:**
  - Diseñá un componente reutilizable llamado TarjetaProducto.
  - Este componente debe ser capaz de mostrar la información esencial de un artículo: imagen, nombre y precio, pasados como props.
- 4. Aplicar estilos de forma modular:**
  - Utilizá un archivo de **CSS global** (como index.css o App.css) para los estilos generales de la página (fuentes, colores de fondo, etc.).
  - Creá un archivo de **CSS Modules** (TarjetaProducto.module.css) para darle estilos específicos y encapsulados únicamente al componente TarjetaProducto.
- 5. Ensamblar la página principal:**
  - En App.jsx, utilizá tu componente Layout para envolver el contenido.
  - Mostrá al menos tres instancias del componente TarjetaProducto con información de ejemplo para simular un catálogo.

### Objetivo del ejercicio práctico:



Este desafío pone a prueba tu capacidad para reutilizar componentes, manejar props y diseñar interfaces dinámicas. ¡Sorpréndenos con tu solución y demuestra por qué TechLab es tu lugar ideal! 🚀

---

### Materiales y recursos adicionales:

- ★ [Documentación oficial de React - Componentes](#)
- ★ [Vite - Getting Started](#)
- ★ [Placeholder.com](#)
- ★ [CSS Tricks - A Complete Guide to Flexbox](#)
- ★ [CSS Tricks - A Complete Guide to Grid](#)

---

### Preguntas para reflexionar:

- ¿Por qué es útil dividir una aplicación en componentes en React?
- ¿Qué papel juegan las props en la personalización de los componentes?
- ¿Cómo ayuda Vite en el desarrollo de aplicaciones React?

---

### Próximos pasos:

- Practicaremos el flujo de datos contenedor-presentacional
- Manejar eventos comunes en React, como clics.
- Aprender a gestionar el estado local de un componente usando el hook useState.
- Crear una funcionalidad básica para agregar productos a un carrito de compras.





**Buenos Aires**  
*aprende*  
Agencia de Habilidades para el Futuro

**BA** Buenos  
Aires  
Ciudad